



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

FairCode: Sviluppo e Validazione di uno Strumento per la Mitigazione dei Bias in Machine Learning

RELATORE

Prof. Fabio Palomba

Dott.ssa Alessandra Parziale

Dott. Gianmario Voria

Università degli Studi di Salerno

CANDIDATO

Marco Greco

Matricola: 0512114908

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Dedica o citazione

"Your focus determines your reality"

- Star Wars: Episode I – The Phantom Menace

Abstract

L'adozione sempre più diffusa di sistemi di Machine Learning in ambiti decisionali critici, come il credito o la giustizia, ha sollevato importanti preoccupazioni etiche legate alla fairness dei modelli predittivi. La letteratura ha proposto diverse metriche e tecniche per misurare e mitigare i bias, ma tali strumenti risultano spesso difficili da applicare nella pratica, poiché richiedono conoscenze specifiche e l'utilizzo di componenti esterni al normale flusso di lavoro degli sviluppatori. In questa tesi è stato progettato e realizzato un software in grado di assistere lo sviluppatore nel calcolo e nella mitigazione dei bias nei progetti di Machine Learning, attraverso un'interfaccia guidata che non richiede competenze avanzate. Il sistema integra un Large Language Model per facilitare l'interazione, offrendo funzionalità come l'analisi del codice e la generazione automatica di snippet personalizzati per l'applicazione di funzioni di mitigazione dei bias. La sperimentazione condotta su dataset benchmark ha evidenziato la capacità del sistema di ridurre le disuguaglianze tra gruppi protetti senza compromettere significativamente l'accuratezza dei modelli, dimostrandone così l'efficacia come strumento di supporto allo sviluppo di soluzioni di Machine Learning più eque.

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Contesto Applicativo	1
1.2 Motivazioni e Obiettivi	2
1.3 Risultati Ottenuti	2
1.4 Struttura Della Tesi	3
2 Background e stato dell'arte	4
2.1 Fairness Nel Machine Learning	4
2.2 Bias	5
2.3 Discriminazione	7
2.4 Metriche	7
2.5 AIF360	9
2.6 Tecniche di Mitigazione dei Bias	9
2.6.1 Tecniche Pre-Processing	10
2.6.2 Tecniche In-Processing	11
2.6.3 Tecniche Post-Processing	12

3	FairCode: Un plugin per la fairness	13
3.1	Obiettivo e Domande Di Ricerca	13
3.2	Architettura Del Sistema	14
3.2.1	Struttura e Interfaccia Utente	15
3.2.2	Funzioni Implementate	16
3.2.3	Gestione Asincrona e Threading	18
3.2.4	Assistente Intelligente	18
3.2.5	Comunicazione tra Plugin e Script Python	20
3.3	Validazione del sistema	21
3.3.1	Validazione della funzione di calcolo delle metriche	21
3.3.2	Validazione della Funzione Reweighing	27
3.3.3	Validazione Prejudice Remover	32
3.3.4	Validazione Funzione di Generazione Snippet	35
4	Risultati	37
4.1	Risultati Per la Funzione di Calcolo	37
4.2	Risultati per la funzione di Reweighing	38
4.3	Risultati della Funzione Prejudice Remover	39
4.4	Risultati della Generazione di Snippet Personalizzati	40
4.5	Considerazioni Finali	41
5	Conclusioni	42
	Bibliografia	43

Elenco delle figure

2.1	Ciclo di retroazione tra bias nei dati, negli algoritmi e nel comportamento degli utenti. Adattato da [1].	5
3.1	Action Buttons	15
3.2	TextArea	16
3.3	Metrics.py	16
3.4	Reweighting.py	17
3.5	Prejudice_Remove.py	17
3.6	AnalyzeButton	18
3.7	Intent_Dataset	19
3.8	Predict_Intent	19
3.9	assistPrompt	20
3.10	Download dataset Adult	22
3.11	Pulizia dataset	22
3.12	Pre processo dataset	23
3.13	salvataggio dataset	23
3.14	caricamento dataset	24
3.15	conversione dataset	24
3.16	calcolo metriche	25
3.17	Pre processo dataset COMPAS	26

3.18 Salvataggio Dataset	26
3.19 Calcolo Metriche COMPAS	27
3.20 Caricamento Pre Reweighing	28
3.21 Metriche Pre Script	29
3.22 Script Reweighing	29
3.23 Metriche Post Script	29
3.24 Caricamento COMPAS	30
3.25 Metriche Pre COMPAS	31
3.26 Reweighing COMPAS	31
3.27 Metriche Post	31
3.28 Caricamento COMPAS	32
3.29 Riconoscimento Gruppi	33
3.30 Split COMPAS	33
3.31 caricamento German	34
3.32 Scaling German	34
3.33 Split German	35
3.34 Helper e calcolo metriche	35

Elenco delle tabelle

4.1	valori delle metriche di fairness	38
4.2	valori delle metriche post reweighing	38
4.3	valori delle metriche $\text{ETA} = 00$	39
4.4	valori delle metriche $\text{ETA} = 25$	39

CAPITOLO 1

Introduzione

1.1 Contesto Applicativo

Negli ultimi anni la diffusione e l'utilizzo di sistemi di Machine Learning (ML) in ambiti decisionali critici ha trasformato il modo in cui vengono prese le decisioni in ambienti come la finanza, la giustizia penale, l'istruzione e la sanità; sollevando preoccupazioni riguardanti l'imparzialità e la fairness dei sistemi in questione. Recenti studi hanno dimostrato che i sistemi di Machine Learning tendono a ereditare e ad ampliare i bias presenti nei dataset di addestramento, portando a discriminazioni ingiuste nei confronti di gruppi svantaggiati. Le principali soluzioni proposte nello stato dell'arte sono le varie metodologie che si categorizzano in base al momento nel ciclo di vita di creazione dei sistemi di Machine Learning in cui vengono applicate, dividendole in tecniche di pre-processing che mirano a eliminare i bias contenuti nei dati di training, tecniche in-processing che modificano direttamente gli algoritmi per raggiungere la fairness e tecniche post-processing che agiscono sulle previsioni che i modelli producono adattandole per eliminare i pregiudizi. Sfortunatamente tutte queste tecniche hanno limitazioni ben precise; nelle tecniche pre-processing la modifica del dataset rischia di alterare la distribuzione dei dati compromettendo le performance del modello, l'in-processing ha limitazioni su modelli black box e le

modifiche potrebbero rendere i modelli meno interpretabili e il post-processing può ridurre la precisione complessiva del modello.

1.2 Motivazioni e Obiettivi

La ricerca nel campo della fairness per i sistemi di Machine Learning ha evidenziato quanto sia importante sviluppare software che siano liberi da discriminazioni e che mantengano delle performance ottimali di predizione. Il lavoro svolto quindi si propone di affrontare queste sfide tramite l'applicazione di modelli di Machine Learning sviluppando un modello capace di supportare uno sviluppatore IA nella creazione di un modello che raggiunga ottimi valori di fairness. L'obiettivo principale quindi è quello di creare un modello di Machine Learning che applichi varie tecniche di unbiasing dei dati e fair Machine Learning per aiutare uno sviluppatore nell'ottimizzazione in termini di fairness del suo modello.

1.3 Risultati Ottenuti

I risultati del lavoro di tesi evidenziano l'efficacia del sistema sviluppato nell'assistere gli sviluppatori nell'analisi e nella mitigazione di unfairness nei modelli di Machine Learning. Il software realizzato consente di calcolare automaticamente metriche di fairness quali Statistical parity difference [2], Disparate Impact [3] e consistency [2], oltre a dare un'analisi tramite IA del codice utente e la possibilità di applicare le tecniche di mitigazione quali Reweighting [3] e Prejudice Remover [?]. I test su dataset standard del dominio hanno dimostrato che l'applicazione assistita dal software di queste tecniche comporta una riduzione delle metriche di bias pur mantenendo un'accuratezza predittiva accettabile. Nel complesso, l'interfaccia interattiva e l'impiego di un classificatore per guidare le risposte dell'IA hanno migliorato l'usabilità dello strumento; in particolare, l'utilizzo di un chatbot che possa analizzare il codice e suggerire snippet personalizzati ha reso il software adatto anche a sviluppatori non esperti in ambito di fairness, dimostrando che il sistema prodotto è un valido supporto alla costruzione di pipeline Machine Learning eque.

1.4 Struttura Della Tesi

La tesi è organizzata come segue:

- **Capitolo 2: Background e stato dell'arte** Vengono introdotti i concetti fondamentali legati alla fairness dei modelli di Machine Learning, vengono descritte le varie metriche utilizzate per calcolare il bias e delle tecniche di mitigazione degli stessi
- **Capitolo 3: FairnessPlugin** Descrive nel dettaglio il sistema realizzato: un plugin per ambiente Pycharm che supporta lo sviluppatore nell'analisi e nella mitigazione della unfairness dei modelli predittivi, dopo aver definito le domande di ricerca, il capitolo illustra l'architettura, l'integrazione dell'LLM utilizzato, il funzionamento dell'analisi del codice e l'attivazione dei moduli Python che applicano le funzioni di mitigazione.
- **Capitolo 4: Risultati e validazione** Presenta la sperimentazione condotta sul software per valutare l'efficacia del sistema e il suo impatto sulle prestazioni tramite l'uso di dataset di benchmark e analizzando i risultati dei calcoli delle metriche di fairness prima e dopo l'applicazione delle funzioni di mitigazione
- **Capitolo 5: Conclusioni e sviluppi futuri** In questo capitolo vengono riassunti i risultati raggiunti, descrivendo il contributo offerto dal plugin in termini di supporto allo sviluppo di modelli equi, vengono poi proposti possibili sviluppi e implementazioni di ulteriori funzioni di mitigazione al software

Background e stato dell'arte

2.1 Fairness Nel Machine Learning

Il concetto di fairness (equità) nel contesto del Machine Learning si riferisce alla capacità di un modello di compiere decisioni che non siano ingiustamente influenzate da attributi protetti o sensibili quali, ad esempio, l'età, il sesso, l'etnia [3, 1]. Nel contesto algoritmico la fairness viene definita in termini formali mediante definizioni matematiche che cercano di catturare il concetto del trattamento equo in un determinato contesto, ma queste varie definizioni sono incompatibili tra loro; la letteratura distingue principalmente due tipi di fairness:

- **Fairness Di Gruppo:** che assicura che determinate statistiche siano simili tra gruppi definiti da attributi sensibili diversi
- **Fairness Individuale:** richiede che individui simili fra loro siano trattati in modo simile dal modello che li esamina

Bilanciare queste due prospettive di equità è molto difficile dato che garantire una potrebbe significare la compromissione dell'altra [4, 5, 6]. Uno dei problemi principali legati alla fairness in ambito di Machine Learning è che anche un modello tecnicamente perfetto può presentare unfairness data dai bias che si presentano sui dati su cui è

addestrato il modello [1] [2]. Negli ultimi anni sono stati proposti numerosi approcci per quantificare e migliorare la fairness nei modelli di Machine Learning; tuttavia, la mancanza di una definizione universale condivisa e l’impatto sulle prestazioni che ha l’ottimizzazione della fairness sulla performance predittiva rendono il problema tuttora aperto e di estrema importanza. In questo contesto, il progetto descritto in questa tesi si inserisce nell’ambito dei tool a supporto per la fairness, per facilitare l’adozione di pratiche e di metriche di equità direttamente nel flusso di lavoro degli sviluppatori.

2.2 Bias

Il termine Bias in ambito Machine Learning non si riferisce solo ad un errore statistico ma assume un significato etico e sociale: una distorsione sistematica dei dati o negli algoritmi che può condurre ad unfairness [1, 3]. Il bias può manifestarsi in diverse fasi del ciclo di vita di un modello come mostrato dalla figura 2.1: dalla fase di raccolta dati, elaborazione, addestramento del modello e interazione utente con il modello, questo fenomeno genera un feedback loop tra dati, algoritmi e comportamenti utente che può amplificare le disuguaglianze [1].

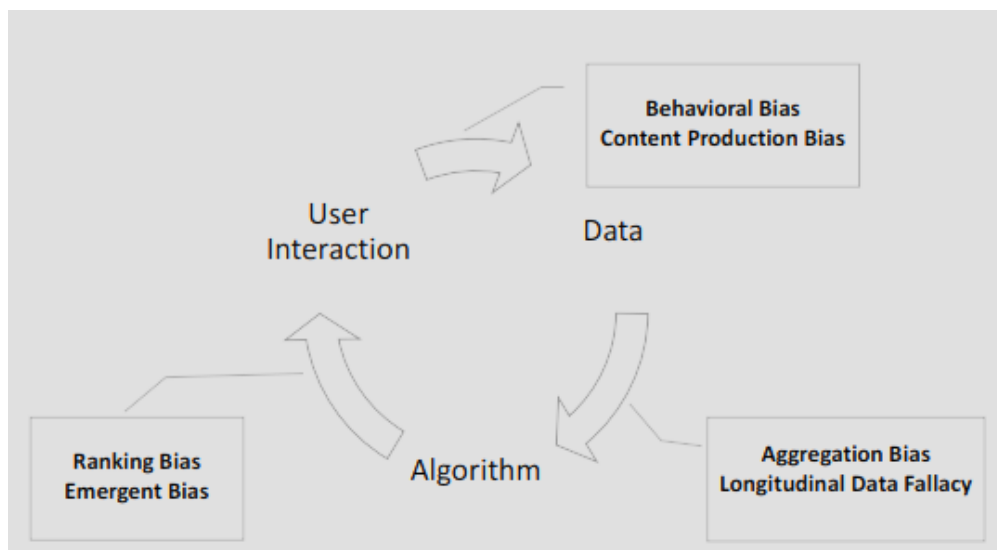


Figura 2.1: Ciclo di retroazione tra bias nei dati, negli algoritmi e nel comportamento degli utenti. Adattato da [1].

La letteratura ha categorizzato diverse tipologie di bias:

- **Bias Nei Dati**

- **Measurement Bias**: distorsione causata da proxy inappropriati [7]
- **Sampling Bias**: deriva da campionamenti non rappresentativi della popolazione target
- **Representation Bias**: dovuto alla sotto rappresentazione di certi gruppi di dati
- **Aggregation Bias**: si verifica quando un modello ignora la varianza tra sottogruppi, assumendo che tutti i sottogruppi possano essere trattati allo stesso modo [8]

- **Bias Algoritmico**

- **Algorithmic Bias**: introdotto dalle scelte di design o dalle funzioni obiettivo del modello, indipendente dal dataset [6]
- **Evaluation Bias**: si verifica quando si utilizzano benchmark non equi o dataset di test squilibrati

- **Bias Da Interazione Utente**

- **User Interaction Bias**: influenzato da interfacce o comportamenti osservabili [9]
- **Popularity Bias**: elementi popolari vengono raccomandati di più, lasciando meno spazio a contenuti non popolari

- **Bias Storico e Sociale**

- **Historical Bias**: riflette le disuguaglianze preesistenti nella società, anche se i dati sono stati raccolti in modo corretto [10]
- **Social Bias**: comportamenti o pregiudizi sociali trasferiti nei dati attraverso annotazioni, interazioni o linguaggio

L'identificazione e la mitigazione dei bias sono la parte fondamentale per lo sviluppo di sistemi equi, tuttavia la loro rimozione completa è spesso impossibile ed è quindi necessario, oltre alle tecniche di bias mitigation, rendere i sistemi trasparenti.

2.3 Discriminazione

Nel contesto del Machine Learning, la discriminazione si riferisce a trattamenti ingiusti o svantaggiosi nei confronti di individui o gruppi protetti a causa del modo in cui il modello è addestrato o applicato; a differenza dei bias, la discriminazione è la conseguenza di una decisione algoritmica ingiusta [1]. Nella letteratura vengono distinti due tipi di discriminazioni:

- **Discriminazione Diretta:** si verifica quando l'attributo protetto influisce in modo diretto sull'esito della decisione. [11]
- **Discriminazione Indiretta:** accade quando l'attributo protetto non è usato direttamente ma è correlato a una variabile proxy. [1]
- **Discriminazione Spiegabile:** è giustificata attraverso variabili legittime. [12]
- **Discriminazione Non Spiegabile:** è ingiustificata sulla base di ogni variabile legittima, va rilevata e rimossa per evitare comportamenti illegali o eticamente inaccettabili.
- **Discriminazione Sistemica:** radicata nelle strutture sociali o nelle organizzazioni [1].
- **Discriminazione Statica:** avviene quando un sistema fa affidamento su medie di gruppo per giudicare soggetti individuali [6]

La discriminazione algoritmica può manifestarsi anche in assenza di intenzionalità da parte dei progettisti; per questo motivo il sistema proposto è volto ad aiutare lo sviluppatore ad applicare in modo semplice e veloce tecniche di mitigazione e di ottimizzazione di fairness.

2.4 Metriche

Per la corretta valutazione di modelli predittivi in ambito di fairness è necessario adottare delle metriche formali che quantifichino le disparità tra vari individui o gruppi, nella letteratura vengono proposte numerose metriche categorizzate in tre macro-categorie [2, 1]:

- **Metriche Di Gruppo**

confrontano le prestazioni del modello tra gruppi distinti definiti da attributi protetti

- **Disparate Impact:** è il rapporto tra le stesse probabilità calcolato con:

$$DI = \frac{P(\hat{Y} = 1 \mid A = 1)}{P(\hat{Y} = 1 \mid A = 0)}$$

Dove $\hat{Y}$ rappresenta la predizione del calcolatore e A rappresenta l'attributo protetto binario, un valore tra 0.8 e 1.25 è spesso considerato accettabile secondo la regola del'80% [13].

- **Statistical Parity Difference:** è una metrica che misura la differenza nella probabilità di ricevere un esito positivo da parte del modello, valuta quanto la distribuzione delle predizioni favorevoli sia equamente bilanciata tra i gruppi e viene calcolata con:

$$SPD = P(\hat{Y} = 1 \mid A = 0) - P(\hat{Y} = 1 \mid A = 1)$$

Dove $\hat{Y}$ è la predizione del calcolatore e A l'attributo protetto binario, un valore prossimo allo zero indica che il modello assegna esiti positivi in modo equo, indipendentemente dal gruppo di appartenenza, viceversa con un numero lontano dallo zero la distribuzione degli esiti positivi non sarà equa tra i diversi gruppi [14].

- **Metriche Individuali**

valutano la validità della predizione del modello tra individui simili

- **Consistency:** verifica se individui simili ricevono predizioni simili, è stimata calcolando la variazione delle predizioni tra vicini nel dataset:

$$\text{Consistency} = 1 - \frac{1}{n} \sum_{i=1}^n \frac{1}{k} \sum_{j \in \text{NN}_k(i)} |\hat{y}_i - \hat{y}_j|$$

La consistency può assumere un valore tra 0 e 1, se il valore restituito tende a 1 allora vuol dire che il modello assegna predizioni identiche a tutti gli individui simili, viceversa un valore tendente a 0 indica che gli individui simili ricevono predizioni differenti segnalando un comportamento ingiusto [14].

Il progetto descritto propone un sistema integrato per il calcolo automatico delle metriche di fairness basato sulla libreria AIF360. In particolare, il plugin fornisce un'interfaccia grafica semplificata e un chatbot di aiuto che permettono all'utente di calcolare le metriche di fairness in modo facile e intuitivo.

2.5 AIF360

L'AI Fairness 360 è un framework open-source sviluppato da IBM Research per aiutare i data scientist e sviluppatori a rilevare, misurare e mitigare i bias nei loro modelli di machine learning. Rilasciato nel 2018, AIF360 è oggi una delle librerie più adottate nella ricerca e nello sviluppo di soluzioni software eque e rappresenta uno standard de facto per l'analisi dell'equità nei sistemi intelligenti. AIF 360 fornisce una vasta gamma di funzioni per la misurazione di metriche di fairness, numerose tecniche di mitigazione e dataset di benchmark integrati [15]. Il toolkit è implementato in Python e si basa su un'architettura modulare; le sue componenti principali sono:

- **BinaryLabelDataset:** una struttura dati centrale di che rappresenta dataset supervisionati binari, annotati con etichette e attributi sensibili.
- **ClassificationMetric:** modulo di calcolo di oltre 70 metriche di fairness
- **Algoritmi Di Mitigazione:** classi dedicate alla mitigazione dei bias tramite tecniche elencate in precedenza

Nel progetto presentato, AIF360 è stato adottato per la sua versatilità e integrazione facilitata; è stato utilizzato per costruire oggetti BinaryDataset a partire dai dati forniti dall'utente, per calcolare alcune metriche chiave di fairness e per la mitigazione di bias in modo immediato e facilitato anche per gli utenti non esperti.

2.6 Tecniche di Mitigazione dei Bias

La letteratura ha sviluppato numerose tecniche per la riduzione dei bias nei sistemi di Machine Learning. Tali metodi, vengono denominati come tecniche di mitigazione, rappresentano strumenti fondamentali per garantire che gli algoritmi

non riproducano, creino o amplifichino disuguaglianze e discriminazioni sociali. Un approccio consolidato in letteratura classifica in tre categorie principali queste tecniche: pre-processing, in-processing e post-processing [3][1][2]

- **Pre-Processing:** Le tecniche pre-processing intervengono sui dati prima dell'addestramento, modificandone la distribuzione o la rappresentazione in modo da ridurre la dipendenza da attributi sensibili [11][13]. L'idea è che, rendendo il dataset più equo, anche un modello standard addestrato su di esso produrrà predizioni più eque[1].
- **In-Processing:** Le tecniche in-processing agiscono durante la fase di addestramento dei modelli, introducendo vincoli o termini di regolarizzazione che penalizzano la dipendenza tra le predizioni e gli attributi sensibili [16, 17]. Questi approcci premettono di ottenere modelli intrinsecamente fairness-aware, ma richiedono l'accesso diretto al processo di ottimizzazione [3][2].
- **Post-processing:** Le tecniche post-processing operano a valle dell'addestramento, modificando le decisioni finali del modello per soddisfare specifici criteri di equità, ad esempio equalizzando i tassi di errore tra gruppi [4]. Questi metodi sono particolarmente utili in scenari black-box, dove non è possibile agire né sui dati né sul training [1][3].

Questa classificazione, consolidata nello stato dell'arte, permette di analizzare i vantaggi e i limiti di ciascuna tecnica in relazione al momento in cui interviene nella pipeline di Machine Learning. Nei paragrafi successivi vengono illustrate alcune tecniche rappresentative per ciascuna categoria.

2.6.1 Tecniche Pre-Processing

Reweighting

Il metodo, ribilancia il dataset attraverso l'assegnazione di pesi alle istanze [11][12]. L'obiettivo è correggere la distribuzione osservata dei dati in modo che rifletta una situazione equa rispetto all'attributo protetto. Il peso di ciascuna istanza è calcolato

come:

$$w(x, y, a) = \frac{P(Y = y) P(A = a)}{P(Y = y, A = a)}$$

dove Y rappresenta l’etichetta e A l’attributo sensibile. In questo modo, i casi appartenenti a gruppi svantaggiati con esito favorevole ricevono maggiore rilevanza, compensando la loro rappresentazione. Reweighing ha dimostrato di migliorare metriche come Statistical Parity Difference e Disparate Impact, senza alterare i dati grezzi [3][1].

Disparate Impact Remover

Questa tecnica modifica le feature non sensibili correlate con attributi protetti, riducendo la possibilità che queste fungano da proxy [13]. Questo metodo trasforma la distribuzione delle variabili in modo da uniformarla tra i diversi gruppi, mantenendo al contempo l’informatività delle feature rispetto al compito predittivo. Pur potendo comportare una riduzione dell’accuratezza, rappresenta un approccio efficace per limitare gli effetti di disparità indiretta nei dati.

2.6.2 Tecniche In-Processing

Prejudice Remover

È un metodo basato su regressione logistica che integra un termine di regolarizzazione nella funzione obiettivo. Tale termine penalizza la dipendenza tra le predizioni e l’attributo protetto, riducendo in modo progressivo il pregiudizio nel modello[16]. La funzione di costo assume la forma:

$$L = - \sum_i \log P(y_i | x_i) + \eta I(\hat{Y}; A)$$

dove $I(\hat{Y}; A)$ rappresenta l’informazione mutua tra predizione e attributo protetto, e η regola l’intensità del vincolo. All’aumentare di η , il modello produce risultati più equi, a fronte di una potenziale perdita in accuratezza. Questo approccio è coerente con il paradigma della fairness through awareness, ovvero un approccio che afferma che un algoritmo è equo se tratta in modo simile individui simili, definendo una distanza tra persone basata su caratteristiche rilevanti [5].

Adversarial Debiasing

Utilizza un approccio ispirato all'“adversarial learning”. In fase di training vengono addestrati due modelli:

- un predittore principale che massimizza l'accuratezza sul compito.
- un discriminatore avversario che cerca di indovinare l'attributo sensibile dalle rappresentazioni apprese.

La funzione di ottimizzazione è definita come:

$$\min_{\theta} \max_{\phi} L_{\text{task}}(\theta) - \lambda L_{\text{adv}}(\theta, \phi)$$

dove L_{task} è la loss del compito principale e L_{adv} quella del discriminatore. In questo modo, il predittore è incentivato a imparare rappresentazioni predittive che non contengano informazione relativa agli attributi protetti [1][3].

2.6.3 Tecniche Post-Processing

Equalized Odds Postprocessing

Questo metodo di debiasing interviene sulle predizioni del modello per garantire che i tassi di veri positivi (TPR) e falsi positivi (FPR) siano equivalenti tra i diversi gruppi [4]. L'algoritmo applica una correzione probabilistica delle decisioni, calibrando le soglie di classificazione in base all'attributo sensibile e al valore reale della classe. Questa tecnica risulta utile in scenari in cui il modello non è modificabile [1].

Il panorama delle tecniche di mitigazione dimostra come non ci sia una soluzione universale ottimale e che la scelta dipende sempre dal contesto applicativo, dal grado di accesso al modello e dal compromesso tra accuratezza ed equità [18][19]. Il software proposto implementa due tecniche di mitigazione dei bias, Reweighting e Prejudice Remover, che rappresentano due approcci complementari: la prima agisce in pre-processing sui dati, mentre la seconda modifica la funzione obiettivo in-processing.

FairCode: Un plugin per la fairness

3.1 Obiettivo e Domande Di Ricerca

L'obiettivo di questo lavoro è sviluppare un sistema che semplifichi l'utilizzo di tecniche di analisi e ottimizzazione della fairness nei modelli di machine learning degli utenti, anche in assenza di competenze specifiche nell'ambito. Nonostante la presenza di numerose librerie come AIF360, l'applicazione di queste tecniche rimane tutt'ora complessa e poco integrata nei flussi di lavoro reali. Per rispondere a questa esigenza, la tesi propone lo sviluppo di un plugin integrato nell'ambiente di Pycharm, in grado di calcolare automaticamente metriche di fairness, presentare risultati in modo chiaro e di guidare tramite un chatbot all'applicazione di tecniche di ottimizzazione di fairness. Da questo obiettivo emergono alcune domande di ricerca fondamentali che guidano la valutazione del progetto.

RQ 1: È possibile supportare gli sviluppatori nella valutazione della fairness dei modelli predittivi direttamente all'interno dell'ambiente di sviluppo?

Questa domanda intende verificare se strumenti integrati come il plugin proposto, possano realmente aiutare gli sviluppatori nell'identificazione, calcolo e correzione di possibili disuguaglianze nei loro modelli.

RQ 2: Un plugin con funzioni preconfigurate e dotato di un'interfaccia testuale guidata può rendere più accessibile il calcolo e l'interpretazione delle metriche di fairness?

Ci si interroga sulla capacità del sistema di mostrare all'utente in modo intuitivo e comprensibile valori di metriche di fairness, favorendo una lettura immediata anche da parte di chi non ha familiarità con il concetto di fairness.

3.2 Architettura Del Sistema

L'architettura del sistema proposto è organizzata secondo un approccio a due livelli principali:

- **Frontend e gestione del plugin:** Implementato con Java e integrato nativamente in Pycharm, gestisce:
 - Interfaccia utente
 - Caricamento del dataset utente
 - Richieste tramite action button e chat
 - Invocazione del modello Gemini
 - Raccolta degli output
 - Download su storage locale dei dataset modificati
- **Motore Analitico:** Implementato in Python e delegato a gestire:
 - Calcolo delle metriche di fairness
 - Mitigazioni in pre-processing

- Mitigazioni in-processing
- Intent Classifier per l'interpretazione delle richieste dell'utente

L'interazione tra i due livelli avviene tramite la chiamata a processi esterni, sfruttando la classe `ProcessBuilder` di Java.

3.2.1 Struttura e Interfaccia Utente

Il plugin fornisce una finestra tool dedicata, composta da:

- **Sezione Superiore:** Una sezione superiore in cui sono presenti una serie di pulsanti per l'esecuzione delle funzioni come mostrato dalla figura 3.1 chiave.

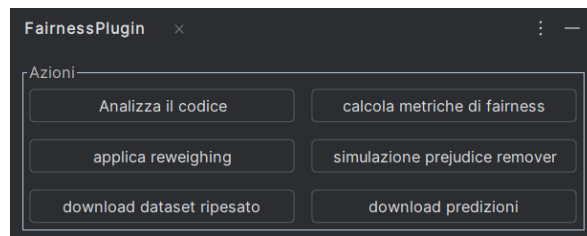


Figura 3.1: Action Buttons

- **Sezione Inferiore:** Una sezione inferiore in cui sono presenti una `JTextArea` usata come chat interattiva tra l'utente e il bot integrato e un campo di testo per l'invio di comandi in linguaggio naturale come mostrato nella figura 3.2.

```

probabilità di predire 1 per ogni gruppo. Più vicino a 0
è, meglio è.
}
metric_frame = MetricFrame(
    metrics=metric_fns,
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=A_test,
)

print(metric_frame.overall)
print(metric_frame.by_group)
'''

Questo esempio di codice modificato include una
valutazione di base della fairness utilizzando la metrica
'demographic_parity_difference'. Questo è solo un punto
di partenza. Un'analisi completa della fairness
richiederebbe l'esplorazione di diverse metriche,
l'applicazione di tecniche di mitigazione del bias e
un'attenta considerazione del contesto specifico del
problema. Ricorda che non esiste una soluzione unica per
il problema della fairness. La scelta delle metriche e
delle tecniche appropriate dipenderà dalle specifiche del
problema e dai valori etici che si vogliono proteggere.

```

calcola le metriche di fairness

Invia

Figura 3.2: JTextArea

3.2.2 Funzioni Implementate

Il plugin sviluppato include un insieme di funzionalità per l'analisi e la mitigazione del codice utente; tutte queste funzionalità sono accessibili sia tramite i pulsanti dedicati nella sezione superiore sia tramite comandi nella JTextArea. Le funzionalità implementate sono le seguenti:

- **Calcolo Delle Metriche Di Fairness:** Questa funzione attivata dallo script `metrics.py` mostrato nella figura 3.3 sfrutta AIF360 per calcolare tre metriche chiave:
 1. Disparate Impact
 2. Statistical Parity Difference
 3. Consistency

```

# Calcolo metriche
metric = BinaryLabelDatasetMetric(dataset,
    privileged_groups=[[{"protected_attr": 1}],
    unprivileged_groups=[[{"protected_attr": 0}]]

print("[METRICHE FAIRNESS]")
print(f"Differenza nella parità statistica: {metric.statistical_parity_difference()}")
print(f"Impatto disparato: {metric.disparate_impact()}")
print(f"Consistenza: {metric.consistency()}")

```

Figura 3.3: Metrics.py

- **Analisi del codice:** Il plugin consente di analizzare il codice attualmente aperto nell'IDE, leggendo il contenuto del file attivo e inviandolo all'assistente LLM per una descrizione e valutazione automatica.
- **Applicazione Reweighing:** Questa funzione avvia lo script `reweighing.py` mostrata in figura 3.4, che ripesa il dataset simulando la funzione ma non applicandola direttamente al codice utente. Il risultato verrà successivamente salvato in un nuovo file `dataset_ripesato.csv` che potrà essere scaricato tramite l'apposito pulsante.

```

re = Reweighing(
    protected_attr=[protected_attr: 0],
    sensitive_attr=[protected_attr: 0])

#calcola le probabilità condizionate tra l'attributo protetto e la label costruisce una matrice di pesi
#basata su P(group) * P(label)/P(group,label)
re.fit(dataset)

#calcola i pesi calcolati da -fit per ogni elemento del dataset crea un nuovo dataset ReweighDataset
#in questo ReweighDataset ogni elemento ha il suo peso calcolato bilanciando la rappresentazione dei gruppi protetti rispetto al non
#distante sotto-rappresentato = peso maggiore
#distante sovra-rappresentato = peso minore
transformed_dataset = re.transform(dataset)

```

Figura 3.4: Reweighing.py

- **Applicazione Prejudice Remover:** Esegue lo script `prejudice_remover.py` mostrato in figura, che applica la tecnica omonima che regolarizza il codice producendo un nuovo dataset di predizioni salvato in `fair_prediction.csv`.

```

#verifica l'attribuzione delle classi come prima ma sul set di training
train_df, _ = train.convert_to_dataframe()
for val in group_vals:
    subset = train_df[train_df[protected_attr] == val]
    label_counts = Counter(subset[label_name])
    print(f"train gruppo {protected_attr} = {val} -> label: {label_counts}")
    if len(label_counts) < 2:
        print(f"errore nel gruppo {val} nel train c'è una sola classe")
        exit(1)

#il parametro eta controlla quanto fortemente il modello penalizza la discriminazione
#con eta 0.0 nessuna penalizzazione e man mano che sale viene dato più peso alla fairness ma con un calo di accuratezza
model = PrejudiceRemover(sensitive_attr=protected_attr, eta=25.0)
try:
    model.fit(train)
except Exception as e:
    print(f"errore durante addestramento: {e}")
    exit(1)

try:
    predictions = model.predict(test)
except Exception as e:
    print(f"errore durante predizione: {e}")
    exit(1)

```

Figura 3.5: Prejudice_Remove.py

- **Generazione di Snippet Personalizzati:** Quando l'utente richiede l'integrazione della tecnica Prejudice Remover nel proprio codice, il plugin invia un prompt personalizzato all'LLM contenente il codice esistente e la richiesta di applicazione della tecnica, il sistema restituisce uno snippet completo e pronto all'uso in Python.

3.2.3 Gestione Asincrona e Threading

Per garantire maggiore fluidità, il plugin è stato progettato in modo da gestire tutte le operazioni computazionalmente complesse in modo asincrono. Per ovviare a eventuali blocchi dell'interfaccia utente, il plugin lancia le operazioni pesanti in thread separati, mantenendo la UI sempre reattiva; questo approccio permette di non caricare l'Event Dispatch Thread, ovvero il thread responsabile di aggiornare l'interfaccia Swing. Nel caso in cui l'esecuzione asincrona debba aggiornare l'interfaccia, viene utilizzato il metodo `SwingUtilities.invokeLater()`, consentendo di effettuare modifiche alla GUI in modo sicuro aggiornando il thread corretto, questa soluzione è adottata sia per le interazioni con gli script Python sia per le chiamate all'API Gemini.

3.2.4 Assistente Intelligente

L'interfaccia utente supporta un'interazione in linguaggio naturale in modo da avere un supporto intelligente all'ottimizzazione che fornisca spiegazioni, risponda a domande generiche e che generi snippet di codice personalizzati per l'utente, il plugin si connette alle API pubbliche di Gemini e adopera il modello Gemini 2.0 Flash tramite una chiamata HTTP POST. Questa connessione è gestita in modo asincrono e supporta:

- l'analisi del codice Python aperto nell'editor tramite `FileEditorManager` mostrato nella figura 3.6

```
analyzeCodeButton.addActionListener(new ActionListener() -> {
    Editor editor = FileEditorManager.getInstance(project).getSelectedTextEditor();
    if (editor != null) {
        String code = editor.getDocument().getText();
        chatArea.append("Codice attuale: " + code + "\n");
        new Thread() -> {
            String prompt = "analizza il codice python e indica se presenta problemi di fairness o bias:\n" + code;
            String botReply = callGemini(systemPrompt, prompt);
            SwingUtilities.invokeLater() -> chatArea.append("bot: " + botReply + "\n\n");
        }.start();
    } else {
        chatArea.append("Codice non trovato\n");
    }
});
```

Figura 3.6: AnalyzeButton

- la costruzione di prompt specifici per il comportamento del codice
- la generazione automatica di risposte tecniche o suggerimenti testuali da parte del LLM.

Per migliorare l'accuracy delle risposte date dal LLM è stato creato un sistema di intent classifier e un relativo dataset. La funzione `Intent_classifier.py` è addestrata su un dataset supervisionato rappresentato in figura 3.7 in cui ogni esempio rappresenta una frase o un comando in linguaggio naturale fornito dall'utente, accompagnato da una classe intent corrispondente.

```
testo,label
mi puoi farmi un'analisi del mio codice?,analizza
mi puoi eseguire reweighing sui dati?,reweighing
puoi calcolare le metriche di fairness?,metriche
riesci a calcolare la fairness del dataset?,metriche
mi puoi ribilanciare i dati con reweighing per favore?,reweighing
mi puoi dirmi quanto è giusto il modello?,metriche
riesci a eseguire reweighing per me?,reweighing
mi puoi analizzare se il mio codice ha bias?,analizza
riesci a analizzare il mio codice?,analizza
puoi analizzare il codice python per me?,analizza
puoi analizzare il mio codice?,analizza
mi puoi analizzare questo script per me?,analizza
riesci a valutare la fairness dei risultati?,metriche
puoi darmi lo snippet per usare prejudice remover?,snippet
mi calcoli le metriche di equità per favore?,metriche
```

Figura 3.7: Intent_Dataset

La funzione mostrata nella figura 3.8 prende quindi in input tutti i comandi digitati dall'utente al bot e restituisce in output una stringa che viene letta dall'LLM e associata a una determinata azione.

```
def predict_intent(user_text):
    """usage: new"""
    if not os.path.exists(MODEL_PATH):
        train_and_save_model()
    if not os.path.exists(MODEL_PATH):
        return
    model = joblib.load(MODEL_PATH)

    #effettua la predizione
    #predict_proba restituisce un array di probabilità per ogni classe
    probabilities = model.predict_proba([user_text])[0]
    max_prob = max(probabilities)
    predict_label = model.classes_[probabilities.argmax()]
```

Figura 3.8: Predict_Intent

Un aspetto rilevante nello sviluppo dell'assistente intelligente è stato l'utilizzo di una variabile `assistPrompt`, progettata per migliorare l'accuratezza e la coerenza degli snippet di codice generati. In particolare, quando l'utente chiede la creazione di uno script che applichi `PrejudiceRemover` di AIF360, il modello linguistico viene guidato da questo prompt che definisce vincoli e linee guida precise come:

- l'obbligo di utilizzare le API corrette della libreria AIF360, evitando funzioni deprecate o parametri errati

- l’inserimento esplicito dei passaggi fondamentali come il caricamento del dataset, definizione dell’attributo protetto, applicazione del transformer e generazione del nuovo dataset
- la produzione di codice autosufficiente ed eseguibile, senza lasciare ambiguità che potrebbero portare a errori di sintassi o runtime

l’uso di `assistPrompt` in figura 3.9, ha permesso di ridurre sensibilmente la variabilità tipica delle risposte generate dagli LLM, ottenendo codice maggiormente stabile, eseguibile senza modifiche e pienamente integrabile nel flusso operativo del plugin.

```
String assistPrompt = """
Sai un assistente che genera SOLO codice Python funzionante da incollare in un progetto ML.
REGOLE FONDAMENTALI:
- Usa PrejudiceRemover da aif360.algorithm.preprocessing (NON preprocessing).
- PrejudiceRemover è IN-PROCESSING: usa pr.fit(train_data) e pr.predict(test_data) (NON transform()).
- Costruisci BinaryLabelDataset passando le variabili/nomi di colonna già presenti nel file assets
  (per il target = "L'azienda è profitto"). Non dimenticare nomi degli attributi già esistenti/variabili equivalenti.
- Usa sempre BinaryLabelDataset con favorable_label=1 e unfavorable_label=0.
- Non rimpiazzare funzioni o costanti esistenti: integra nel flusso attuale con il minimo codice necessario.
- Non usare ROC AUC se hai solo predizioni binarie, limita la valutazione a accuracy e matrice di confusione.
- Non fare le code = trasformazioni manuali alle feature di BinaryLabelDataset, usa il dataset così com'è.
- Per il parametro sta di PrejudiceRemover:
  * se nel file esiste già una costante (es. PR.TRAIDIFF_OMEGA), rimbaltala.
  * altrimenti, dichiara una costante pubblica (es. PR.TRAIDIFF = 25.0) e usala. NON hardcodare un numero dentro il costruttore.
- Restituisci SOLO codice Python eseguibile tra """ python """ senza testo extra.
""" + code;

new Thread() -> {
String reply = callBackend(assistPrompt + "", assistThread);
```

Figura 3.9: `assistPrompt`

3.2.5 Comunicazione tra Plugin e Script Python

La comunicazione con il frontend in Java e gli script Python è stata realizzata tramite l’uso della classe `ProcessBuilder`, che consente di avviare processi esterni dalla JVM e catturarne l’output. Ogni volta che viene richiesta l’esecuzione di una funzionalità che deriva dalla libreria di AIF360, il plugin costruisce un comando contenente:

- lo script Python da eseguire
- il percorso assoluto dello script
- eventuali parametri aggiuntivi presi in input dallo script Python

il plugin cattura l’output in tempo reale eseguendo l’azione richiesta dallo script, la lettura avviene riga per riga tramite un `BufferedReader` collegato allo stream del processo, una volta che l’esecuzione dello script è terminata, il thread associato invia il risultato alla GUI tramite `SwingUtilities.invokeLater()`.

3.3 Validazione del sistema

Per la validazione delle funzionalità implementate nel plugin sono stati utilizzati dei dataset standard, ampiamente adottati nella letteratura. L'impiego di dataset di riferimento consente non solo la riproducibilità degli esperimenti, ma anche la comparabilità con studi precedenti, come evidenziano [3, 1]. In particolare, la comunità scientifica fa uso di tre dataset principali che coprono scenari applicativi differenti e costituiscono di fatto benchmark consolidati [6]

- **Adult (UCI Census Income):** estratto dal Current Population Survey del 1994 [20], è stato impiegato in numerosi studi per analizzare le disparità legate al sesso e alla razza nelle predizioni di reddito [18, 21].
- **COMPAS:** reso celebre per l'inchiesta di ProPublica [7], rappresenta uno dei casi più discussi di bias algoritmo in ambito giudiziario. È stato utilizzato in diversi lavori come banco di prova per le metriche di fairness e per gli algoritmi di mitigazione dei bias [22, 23].
- **German Credit:** originariamente raccolto presso una banca tedesca negli anni Settanta e successivamente ripubblicato nel repository UCI, è ampiamente usato per studiare problemi di scoring creditizio e per analizzare discriminazioni legate all'età e allo status socio-demografico [24, 13, 25].

L'impiego di questi dataset eterogenei ha permesso di testare la correttezza delle funzioni implementate nel plugin e di confrontare i risultati con quelli attesi dalle funzioni della libreria AIF360, considerata uno strumento per l'analisi della fairness di riferimento nei modelli di machine learning [15].

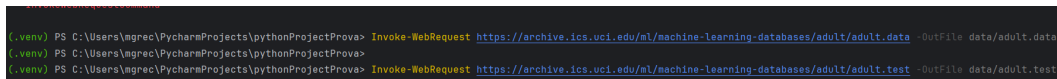
3.3.1 Validazione della funzione di calcolo delle metriche

La prima fase di validazione ha riguardato la funzione di calcolo delle metriche di fairness implementata nel plugin. L'obiettivo è accertare che le metriche Statistical Parity Difference, Disparate Impact e Consistency venissero calcolate correttamente sui dataset standard Adult (UCI) e COMPAS; la validazione è stata fatta calcolando prima le metriche di fairness tramite uno script eseguito dalla Python console e

successivamente il processo di calcolo è stato eseguito tramite l'apposita funzione nel plugin.

Il test eseguito sul dataset Adult (UCI) è stato diviso in diverse fasi:

1. **Preparazione del Dataset:** Il dataset Adult (UCI) è stato scaricato dalla repository UCI Machine Learning, nei file raw `adult.data` e `adult.test` come mostrato in figura 3.10.



```

.venv\ PS C:\Users\ngrec\PycharmProjects\pythonProjectProva> Invoke-WebRequest https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data -OutFile data/adult.data
.venv\ PS C:\Users\ngrec\PycharmProjects\pythonProjectProva> Invoke-WebRequest https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.test -OutFile data/adult.test

```

Figura 3.10: Download dataset Adult

Successivamente, i file raw sono stati importati in Python e uniti in un unico `DataFrame`, questa fase sono state svolte operazioni sul dataset mostrate nella figura 3.11 quali:

- assegnazione dei nomi alle colonne
- eliminazione dei valori mancanti indicati dai simboli `NaN` e `?`
- normalizzazione della colonna `income`, rimuovendo caratteri superflui come punti e spaziature.

In questo modo i file raw vengono ripuliti dalle imperfezioni tipiche dei dataset legacy.



```

import pandas as pd

columns = [
    "age", "workclass", "fnlwgt", "education", "education_num",
    "marital_status", "occupation", "relationship", "race", "sex",
    "capital_gain", "capital_loss", "hours_per_week", "native_country", "income"
]

train = pd.read_csv("data/adult.data", header=None, names=columns, na_values="?", skipinitialspace=True)
test = pd.read_csv("data/adult.test", header=0, names=columns, na_values="?", skipinitialspace=True)

df = pd.concat([train, test], ignore_index=True)

df = df.dropna()
df["income"] = df["income"].str.replace(".", "", regex=False).str.strip()

```

Figura 3.11: Pulizia dataset

2. **Processing e creazione delle variabili numeriche:** Successivamente, il dataset è stato pre processato per renderlo compatibile con AIF360, il codice ha previsto più fasi mostrate nella seguente figura 3.12:

- Creazione della variabile target numerica `income_num` impostata con valore 1 se $>50K$ e 0 se $\leq 50K$
- Creazione delle colonne protette numeriche `sex_num` impostata a 1 per Male o 0 per Female e `race_num` impostata a 1 se White e 0 se non-White
- Applicazione della one-hot encoding a tutte le variabili categoriche restanti, così da trasformarle in variabili binarie
- Assemblaggio del dataset finale includendo feature numeriche, attributi protetti e label
- Sanity Check per verificare che non ci fossero valori nulli e che tutte le colonne risultassero numeriche.

Questa fase è stata cruciale per far interpretare correttamente il dataset alle funzioni AIF360.

```
df["income"] = df["income"].str.replace(".", "", regex=False).str.strip()
df["income_num"] = df["income"].map({">50K": 1.0, "<=50K": 0.0})

df["sex_num"] = df["sex"].map({"Male": 1.0, "Female": 0.0})
df["race_num"] = (df["race"] == "White").astype(float)

categoriche = ["workclass", "education", "marital_status", "occupation",
               "relationship", "native_country"]
numeriche = ["age", "fnlwt", "education_num", "capital_gain",
             "capital_loss", "hours_per_week"]

X = pd.get_dummies(df[categoriche + numeriche], drop_first=True)
proc = pd.concat([X, df[["sex_num", "race_num", "income_num"]]], axis=1)

assert proc.isna().sum().sum() == 0
assert all(proc.dtypes.apply(lambda t: pd.api.types.is_numeric_dtype(t)))
```

Figura 3.12: Pre processo dataset

3. **Salvataggio del dataset pre processato:** Il dataset così restituito è stato salvato in un file CSV denominato `adult_proc.csv` come mostrato in figura 3.13. Questo file rappresenta la versione pre processata stabile che viene riutilizzata sia per i calcoli manuali sia per il confronto con la funzione implementata nel plugin.

```
>>> import os
...
... os.makedirs("tests/fixtures", exist_ok=True)
... df.to_csv("tests/fixtures/adult_proc.csv", index=False)
... print("Salvato in tests/fixtures/adult_proc.csv")
Salvato in tests/fixtures/adult_proc.csv
```

Figura 3.13: salvataggio dataset

4. **Caricamento del dataset pre processato:** Nella fase successiva, il file `adult_proc.csv` è stato caricato in memoria e le colonne principali sono state convertite esplicitamente in formato float come mostrato dalla figura 3.14, così da assicurare l'assenza di errori data dalla tipizzazione dei dati nel passaggio ad AIF360

```
import pandas as pd

DATA_CSV = "tests/fixtures/adult_proc.csv"

print(">>> STEP 1: Caricamento dataset")
df = pd.read_csv(DATA_CSV)
for c in ["income_num", "sex_num", "race_num"]:
    df[c] = df[c].astype(float)

print(f"shape: {df.shape}")
print(f"colonne: {df.columns.tolist()[:10]} ...")
```

Figura 3.14: caricamento dataset

5. **Conversione in BinaryLabelDataset:** Il dataset pre processato è stato trasformato in un oggetto `BinaryLabelDataset` di AIF360 come mostrato nella figura 3.15, in questa fase vengono definiti esplicitamente:

- quale fosse l'etichetta (`income_num`)
- il valore favorevole (1.0) e quello sfavorevole (0.0)
- quali colonne rappresentassero gli attributi protetti (`sex_num`, `race_num`)

Questa rappresentazione standardizzata è il prerequisito per calcolare le metriche di fairness

```
from aif360.datasets import BinaryLabelDataset

dataset = BinaryLabelDataset(
    favorable_label=">50K",
    unfavorable_label="<=50K",
    df=df,
    label_names=["income"],
    protected_attribute_names=["sex", "race"]
)
```

Figura 3.15: conversione dataset

6. **Calcolo delle Metriche:** Una volta costruito il `BinaryLabelDataset`, è stato possibile calcolare manualmente le metriche con AIF360 come mostrato dalla figura 3.16:

- La Consistency globale del dataset, usando `consistency(n_neighbors=5)`
- La Statistical Parity Difference per sesso e razza, intesa come differenza tra i sessi di esito favorevole nei gruppi protetti e non protetti
- Il Disparate Impact, calcolato come rapporto fra gli stessi sessi

Tutti i risultati sono stati salvati in un file di riferimento `ref_metrics.csv`, che rappresenta la baseline per il confronto con il plugin.

```
import os, pandas as pd
from aif360.datasets import BinaryLabelDataset
from aif360.metrics import BinaryLabelDatasetMetric

DATA_CSV = "tests/fixtures/compas_proc.csv"
OUT_DIR = "tests/results"
OUT_CSV = f"{OUT_DIR}/compas_ref_metrics.csv"
os.makedirs(OUT_DIR, exist_ok=True)

df = pd.read_csv(DATA_CSV)
for c in ["two_year_recid", "sex_num", "race_num"]:
    df[c] = df[c].astype(float)

LABEL = "two_year_recid" # favorevole = 0.0
K = 5

bld = BinaryLabelDataset(
    favorable_label=0.0, unfavorable_label=1.0,
    df=df, label_names=[LABEL],
    protected_attribute_names=["sex_num", "race_num"]
)

metric_all = BinaryLabelDatasetMetric(bld, [], [])
cons_val = metric_all.consistency(n_neighbors=K).item()

rows = []
for name, col in [("sex", "sex_num"), ("race", "race_num")]:
    metr = BinaryLabelDatasetMetric(
        bld,
        privileged_groups=[[col: 1.0]],
        unprivileged_groups=[[col: 0.0]]
    )
    spd = float(metr.mean_difference()) # unpriv - priv
    di = float(metr.disparate_impact()) # unpriv / priv (NaN se rate_priv=0)
    rows.append({"attribute": name, "spd": spd, "di": di, "consistency": cons_val, "K": K})

ref = pd.DataFrame(rows)
ref.to_csv(OUT_CSV, index=False)
print(ref.to_string(index=False))
print(f"Salvato in", OUT_CSV)
```

Figura 3.16: calcolo metriche

7. **Test sul Plugin:** successivamente il calcolo delle metriche è stato effettuato tramite l'apposito `actionButton` nell'interfaccia del plugin e sono stati confrontati i risultati.

Analogamente a Adult (UCI) il test eseguito sul dataset COMPAS è stato diviso in più fasi:

1. Pre processing COMPAS: in questa fase importiamo il dataset tramite

CompasDataset di AIF360, lo convertiamo in DataFrame e prepariamo le variabili chiave, il processo è mostrato nella figura 3.17:

- Label `two_year_recid` convertita a numerico float
- Attributi protetti numerici come `sex_num` impostato a 1 se Male altrimenti 0 e `race_num` impostato a 1 se Caucasian altrimenti 0, il mapping esplicito appena illustrato viene fatto per definire in modo chiaro il gruppo privilegiato o non privilegiato.
- One-hot encoding delle categorie residue mantenendo le numeriche originali

```
import os, pandas as pd
from aif360.datasets import CompasDataset

OUT = "tests/fixtures/compas_proc.csv"
os.makedirs("tests/fixtures", exist_ok=True)

ds = CompasDataset()
df, _ = ds.convert_to_dataframe()

df["two_year_recid"] = df["two_year_recid"].astype(float)
df["sex_num"] = (df["sex"] == "Male").astype(float)
df["race_num"] = (df["race"] == "Caucasian").astype(float)

cat_cols = [c for c in df.select_dtypes(include="object").columns if c not in ["sex", "race"]]
num_cols = [c for c in df.select_dtypes(include="object").columns if c not in ["two_year_recid", "sex_num", "race_num"]]

X = pd.get_dummies(df[cat_cols], drop_first=True)
proc = pd.concat([df[num_cols], X, df["sex_num", "race_num", "two_year_recid"]], axis=1)

proc.to_csv(OUT, index=False)
print("OK -> OUT, proc shape")
```

Figura 3.17: Pre processo dataset COMPAS

2. Caricamento del dataset pre processato e cast dei tipi: Viene richiamato il file `compas_proc.csv` e viene forzato il tipo `float` su `two_year_recid`, `sex_num` e `race_num`. Questo rende la pipeline deterministica, evita i mismatch di tipo e previene errori di AIF360 il processo è mostrato nella figura 3.18.

```
import pandas as pd

DATA_CSV = "tests/fixtures/compas_proc.csv"

print(">>> STEP 1: Caricamento dataset")
df = pd.read_csv(DATA_CSV)
for c in ["two_year_recid", "sex_num", "race_num"]:
    df[c] = df[c].astype(float)

print(f"shape: {df.shape}")
print(f"prime colonne: {df.columns.tolist()[:10]} ...")
```

Figura 3.18: Salvataggio Dataset

3. **Calcolo delle metriche di fairness:** Nella figura 3.19 viene mostrato il processo di calcolo delle metriche che a partire dal file `compas_proc.csv` si costruisce un `BinaryLabelDataset` impostando la semantica corretta della label per COMPAS ovvero la label non recidivo (`favorable_label`) impostata 0.0 e la label recidivo (`unfavorable_label`) impostata a 1.0 e indicando gli attributi protetti `sex_num` e `race_num`. Vengono quindi calcolate Consistency globale con $k = 5$, Statistical Parity Difference e Disparate Impact per ciascun attributo protetto, lo script salva successivamente i valori in un file `compas_ref_metrics.csv` usato come baseline per confrontare l'output del plugin.

```
import os, pandas as pd
from aif360.datasets import BinaryLabelDataset
from aif360.metrics import BinaryLabelDatasetMetric

DATA_CSV = "tests/fixtures/compas_proc.csv"
OUT_DIR = "tests/results"
OUT_CSV = f"{OUT_DIR}/compas_ref_metrics.csv"
os.makedirs(OUT_DIR, exist_ok=True)

df = pd.read_csv(DATA_CSV)
for c in ["two_year_recid", "sex_num", "race_num"]:
    df[c] = df[c].astype(float)

LABEL = "two_year_recid"
K = 5

bld = BinaryLabelDataset(
    favorable_label=0.0, unfavorable_label=1.0, # 0 = no recidive (favorevole), 1 = recidive (sfavorevole)
    df=df, label_names=[LABEL],
    protected_attribute_names=["sex_num", "race_num"]
)
metric_all = BinaryLabelDatasetMetric(bld, [], [])
cons_val = metric_all.consistency(n_neighbors=K).item()

ATTS = {
    "sex": {"col": "sex_num", "priv": 1.0, "unpriv": 0.0},
    "race": {"col": "race_num", "priv": 1.0, "unpriv": 0.0},
}

def rate_pos(d, group_col, group_val):
    s = d[d[group_col] == group_val]
    return float((s[LABEL] == 0.0).mean()) if len(s) else float("nan") # outcome positivo = no recidive

rows = []
for name, spec in ATTS.items():
    r_u = rate_pos(df, spec["col"], spec["unpriv"])
    r_p = rate_pos(df, spec["col"], spec["priv"])
    spd = float(r_u - r_p)
    di = float("nan") if r_p == 0 else float(r_u / r_p)
    rows.append({"attribute": name, "spd": spd, "di": di, "consistency": cons_val, "k": K})

ref = pd.DataFrame(rows)
ref.to_csv(OUT_CSV, index=False)

print(">>> STEP 2: Metriche di riferimento (COMPAS)")
print(ref.to_string(index=False))
print(f"\nSalvato in {OUT_CSV}")
```

Figura 3.19: Calcolo Metriche COMPAS

4. **Test sul Plugin:** Come per Adult (UCI), il calcolo delle metriche è stato eseguito tramite l'interfaccia del plugin e sono stati confrontati i risultati con quelli in output dallo script.

3.3.2 Validazione della Funzione Reweighing

Per verificare l'efficacia della funzione di reweighing implementata nel plugin è stato condotto un esperimento sui dataset Adult UCI e COMPAS. In questo caso, la

validazione è stata condotta su due livelli distinti:

- **Validazione della funzione di mitigazione:** tramite il confronto tra le metriche di fairness calcolate prima e dopo dell'applicazione del reweighing, al fine di verificare che l'algoritmo producesse effettivamente un miglioramento in termini di Statistical Parity Difference e Disparate Impact.
- **Validazione dell'implementazione del plugin:** tramite il confronto tra i risultati ottenuti con l'esecuzione manuale in console e quelli generati dal plugin, sia nella fase pre sia nella fase post reweighing. In questo modo è stato possibile accertare che il plugin richiami correttamente la funzione della libreria, senza introdurre differenze o errori rispetto agli script originali.

Di seguito vengono riportate le fasi relative alle operazioni eseguite per il test sul dataset Adult (UCI):

1. **Caricamento del Dataset e Creazione del BinaryLabelDataset:** Si importa il CSV pre processato e si istanzia `BinaryLabelDataset` mostrato in figura 3.20 indicando:

- Il nome della label `income_num` impostata a 1 per la label favorita e a 0 per quella sfavorita
- i nomi degli attributi protetti `sex_num` `race_num`

```
import pandas as pd
from aif360.datasets import BinaryLabelDataset
from aif360.metrics import BinaryLabelDatasetMetric
from aif360.algorithms.preprocessing import Reweighing

# --- Carica ---
df = pd.read_csv("tests/fixtures/adult_proc.csv")
PROT_ATTRS = ["sex_num", "race_num"]

bld = BinaryLabelDataset(
    favorable_label=1.0, unfavorable_label=0.0,
    df=df, label_names=["income_num"],
    protected_attribute_names=PROT_ATTRS
)
```

Figura 3.20: Caricamento Pre Reweighing

2. **Calcolo delle metriche pre-reweighing:** Per ciascun attributo protetto si crea un `BinaryLabelDatasetMetric` e si calcolano Statistical Parity Difference e Disparate Impact come mostrato nella figura 3.21.

```
for attr in PROT_ATTRS:
    metric = BinaryLabelDatasetMetric(
        bld,
        unprivileged_groups=[{attr: 0.0}],
        privileged_groups=[{attr: 1.0}],
    )
    print(f"[BASE - {attr}] SPD: {metric.mean_difference()}")
    print(f"[BASE - {attr}] DI : {metric.disparate_impact()}")
    print("-"*42)
```

Figura 3.21: Metriche Pre Script

3. **Applicazione del Reweighing:** Si istanzia l'oggetto `Reweighing` e viene chiamato `fit_transform` sul dataset iniziale come mostrato in figura 3.22, che calcola e assegna `instance_weights` a ogni riga.

```
rw = Reweighing(
    unprivileged_groups=[{a: 0.0} for a in PROT_ATTRS],
    privileged_groups=[{a: 1.0} for a in PROT_ATTRS],
)
bld_rw = rw.fit_transform(bld) # bld_rw ha instance_weights
```

Figura 3.22: Script Reweighing

4. **Calcolo metriche post-reweighing:** Nella figura 3.23 viene mostrato il calcolo di Statistical Parity Difference e Disparate Impact con `BinaryLabelDatasetMetric`, le funzioni delle metriche tengono conto dei `instance_weights`, quindi misurano le disparità sul dataset riequilibrato.

```
for attr in PROT_ATTRS:
    metric_post = BinaryLabelDatasetMetric(
        bld_rw,
        unprivileged_groups=[{attr: 0.0}],
        privileged_groups=[{attr: 1.0}],
    )
    print(f"[POST - {attr}] SPD: {metric_post.mean_difference()}")
    print(f"[POST - {attr}] DI : {metric_post.disparate_impact()}")
    print("-"*42)
```

Figura 3.23: Metriche Post Script

5. **Pipeline del plugin:** Successivamente al test svolto tramite script su Python Console, viene eseguita la stessa pipeline sul plugin:

- Viene caricato il dataset Adult (UCI) nel form di input creato alla pressione del' `actionButton` che implementa il calcolo delle metriche di fairness e vengono quindi stampati i risultati del calcolo.
- Successivamente viene avviata tramite la pressione del' `actionButton` che implementa la funzionalità del reweighing, il ripesamento del dataset e viene salvato il nuovo dataset bilanciato
- In fine il dataset bilanciato viene scaricato in memoria locale e caricato nel form di calcolo delle metriche e vengono stampati i nuovi risultati.

La stessa pipeline è stata eseguita per il dataset COMPAS; di seguito vengono riportate le relative fasi:

1. **Caricamento e setup del dataset:** Viene letto `compas_proc.csv` e si costruisce un `BinaryLabelDataset` come mostrato nella figura 3.24 impostando:

- il nome della label `two_year_recid` impostata a 0 per il valore "non recidivo" e 1 altrimenti
- il nome degli attributi protetti `sex_num` e `race_num`

```
import pandas as pd
from aif360.datasets import BinaryLabelDataset
from aif360.metrics import BinaryLabelDatasetMetric

CSV = "tests/fixtures/compas_proc.csv"
LABEL = "two_year_recid"
PROT_ATTRS = ["sex_num", "race_num"]

df = pd.read_csv(CSV)

bld = BinaryLabelDataset(
    favorable_label=0.0,
    unfavorable_label=1.0,
    df=df,
    label_names=[LABEL],
    protected_attribute_names=PROT_ATTRS
)
```

Figura 3.24: Caricamento COMPAS

2. **Metriche pre reweighing:** per ciascun attributo protetto viene istanziato un oggetto `BinaryLabelDatasetMetric` e vengono stampati Statistical Parity Difference e Disparate Impact mostrato in figura 3.25.

```

for attr in PROT_ATTRS:
    metric = BinaryLabelDatasetMetric(
        bld,
        unprivileged_groups=[{attr: 0.0}],
        privileged_groups=[{attr: 1.0}],
    )
    print(f"[BASE - {attr}] SPD: {metric.mean_difference()}")
    print(f"[BASE - {attr}] DI : {metric.disparate_impact()}")
    print("-"*42)

```

Figura 3.25: Metriche Pre COMPAS

3. **Applicazione del Reweighting:** viene istanziato l'oggetto `Reweighting`, e viene applicata la funzione `fit_transform` al dataset come mostrato in figura 3.26 per poi salvare il nuovo dataset bilanciato con la colonna `instance_weights`.

```

from aif360.algorithms.preprocessing import Reweighting
import numpy as np

rw = Reweighting(
    unprivileged_groups=[{a: 0.0} for a in PROT_ATTRS],
    privileged_groups=[{a: 1.0} for a in PROT_ATTRS],
)

```

Figura 3.26: Reweighting COMPAS

4. **Metriche post reweighting:** vengono calcolate Statistical Parity Difference e Disparate Impact sul dataset bilanciato (figura 3.27)

```

from aif360.metrics import BinaryLabelDatasetMetric

for attr in PROT_ATTRS:
    metric_post = BinaryLabelDatasetMetric(
        bld_rw,
        unprivileged_groups=[{attr: 0.0}],
        privileged_groups=[{attr: 1.0}],
    )
    print(f"[POST - {attr}] SPD: {metric_post.mean_difference()}")
    print(f"[POST - {attr}] DI : {metric_post.disparate_impact()}")
    print("-"*42)

```

Figura 3.27: Metriche Post

5. **Pipeline del Plugin:** analogamente al test svolto su Adult (UCI) nella fase 5, viene rieseguita la pipeline nel plugin anche per il dataset COMPAS con le relative label e attributi protetti.

3.3.3 Validazione Prejudice Remover

Per verificare la correttezza dell'implementazione della funzione Prejudice Remover nel plugin è stato condotto un esperimento sui dataset COMPAS e German Credit. Anche in questo caso, la validazione si è articolata su due livelli:

- Validazione della funzione di mitigazione: Avvenuta tramite il confronto tra le metriche di fairness calcolate a partire dal modello standard, ovvero senza vincoli con $\eta = 0$ e da quello regolarizzato con $\eta = 25$, per verificare che l'algoritmo modifichi effettivamente le predizioni di una maggiore equità
- Validazione dell'implementazione del plugin: La verifica che i valori ottenuti dal plugin coincidano con quelli prodotti dall'esecuzione manuale in console, garantendo che il richiamo alla libreria AIF360 sia corretto e privo di differenze rispetto allo script di riferimento.

Di seguito sono riportate le fasi relative al test condotto su COMPAS

1. **Caricamento del dataset e controlli preliminari:** La figura 3.28 mostra come il dataset COMPAS pre processato viene letto da file CSV tramite pandas. Successivamente si eseguono controlli sulle colonne dell'attributo protetto `race_num` e della label `two_year_recid`, verificando la loro presenza, l'assenza di valori mancanti e l'unicità dei valori ammessi.

```
import pandas as pd
import numpy as np
from collections import Counter
from aif360.datasets import StandardDataset
from aif360.algorithms.inprocessing import PrejudiceRemover
from aif360.metrics import ClassificationMetric

CSV_PATH = "tests/fixtures/compas_proc.csv"
PROT_ATTR = "race_num"
LABEL_COL = "two_year_recid"
SEED = 42

df = pd.read_csv(CSV_PATH)
df[PROT_ATTR] = df[PROT_ATTR].astype(float)
df[LABEL_COL] = df[LABEL_COL].astype(float)

print("Shape:", df.shape)
print("Colonne OK?:", {PROT_ATTR: PROT_ATTR in df.columns, LABEL_COL: LABEL_COL in df.columns})
print("NaN prot.:", df[PROT_ATTR].isna().sum(), " | NaN label:", df[LABEL_COL].isna().sum())
print("Uniq prot.:", sorted(df[PROT_ATTR].unique().tolist()))
print("Uniq label:", sorted(df[LABEL_COL].unique().tolist()))
```

Figura 3.28: Caricamento COMPAS

2. **Identificazione dei gruppi privilegiati e non privilegiati:** Tramite una procedura di analisi preliminare vengono calcolati i tassi favorevoli per ciascun gruppo

definito sull'attributo protetto, il gruppo con tasso maggiore viene assunto come privilegiato, quello con tasso minore come non privilegiato (figura 3.29).

```
rates = {}
for g in sorted(df[PROT_ATTR].unique()):
    sub = df[df[PROT_ATTR] == g]
    rates[g] = (sub[LABEL_COL] == 0.0).mean() # tasso di "no recidiva"
priv_val = float(max(rates, key=rates.get))
unpriv_val = float(min(rates, key=rates.get))
print(f"Tasso favorevole per gruppo: {rates} -> privileged={priv_val}, unprivileged={unpriv_val}")
```

Figura 3.29: Riconoscimento Gruppi

3. **Split del dataset in training e test:** La figura 3.30 mostra come il dataset viene suddiviso in sottoinsiemi di addestramento e test secondo una proporzionalità 70/30, assicurando la riproducibilità tramite un seed fisso.

```
dataset = StandardDataset(
    df,
    label_name=LABEL_COL,
    favorable_classes=[0.0],
    protected_attribute_names=[PROT_ATTR],
    privileged_classes=[[priv_val]]
)
print("OK StandardDataset (COMPAS) creato.")

np.random.seed(SEED)
train, test = dataset.split([0.7], shuffle=True)

print("Distribuzione label TRAIN:", Counter(train.labels.ravel()))
print("Distribuzione label TEST :", Counter(test.labels.ravel()))

train_df, _ = train.convert_to_dataframe()
for val in sorted(train_df[PROT_ATTR].unique()):
    cnt = Counter(train_df[train_df[PROT_ATTR] == val][LABEL_COL])
    print(f"[TRAIN] gruppo {PROT_ATTR}={val} -> labels {cnt}")

print("Protected names:", train.protected_attribute_names)
print("Prot values uniq (train):", np.unique(train.protected_attributes))
```

Figura 3.30: Split COMPAS

4. **Calcolo delle metriche con $\eta = 0$:** Si addestra un modello Prejudice Remover senza vincolo di fairness, e si calcolano le metriche Statistical Parity Difference, Disparate Impact, Consistency e Accuracy tramite la classe `ClassificationMetric`.
5. **Calcolo delle metriche con $\eta = 25$:** Si ripete l'addestramento di Prejudice Remover ma aggiungendo i vincoli di fairness volto a ridurre la discriminazione. Vengono poi calcolate le stesse metriche della fase 4.
6. **Pipeline del plugin:** Infine, l'intera pipeline viene replicata all'interno del plugin utilizzando le funzioni implementate e lo stesso dataset e parametri.

Lo stesso procedimento è stato seguito per i test sul dataset German Credit; di seguito vengono riportate le relative fasi:

1. **Caricamento del dataset:** Il dataset German Credit è disponibile direttamente in AIF360 tramite la classe `GermanDataset`. Viene quindi caricato e convertito in `DataFrame` come mostrato in figura 3.31, recuperando automaticamente le informazioni relative alla label, attributi protetti, gruppi privilegiati e non privilegiati.

```
import pandas as pd
from collections import Counter

from aif360.datasets import GermanDataset, StandardDataset
from aif360.algorithms.inprocessing import PrejudiceRemover
from aif360.metrics import ClassificationMetric

SEED = 42

base = GermanDataset()
df, meta = base.convert_to_dataframe()

label_col = base.label_names[0]          # es. "credit"
prot_attr = base.protected_attribute_names[0] # tip. "age" (già 0/1)
fav_label = base.favorable_label         # 1
priv_val = base.privileged_protected_attributes[0][0] # 1
unpriv_val = base.unprivileged_protected_attributes[0][0] # 0
```

Figura 3.31: caricamento German

2. **Pre processing e scaling della variabili numeriche:** Al fine di migliorare la stabilità del modello, le colonne numeriche sono state normalizzate tramite uno scaling robusto. Successivamente è stato ricreato lo `StandardDataset` includendo label, attributo protetto e classi privilegiate (figura 3.32).

```
num_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
for c in [label_col, prot_attr]:
    if c in num_cols:
        num_cols.remove(c)

def robust_scale(s: pd.Series) -> pd.Series:
    med = s.median()
    iqr = s.quantile(0.75) - s.quantile(0.25)
    return (s - med) / iqr if iqr != 0 else (s - med)

df_scaled = df.copy()
for c in num_cols:
    df_scaled[c] = robust_scale(df_scaled[c])

dataset = StandardDataset(
    df_scaled,
    label_name=label_col,
    favorable_classes=[fav_label],
    protected_attribute_names=[prot_attr],
    privileged_classes=[[priv_val]]
)
```

Figura 3.32: Scaling German

3. **Split in training e test set:** La figura 3.33 mostra come il dataset viene diviso in training e test set con una proporzione di 70/30, mantenendo la corretta distribuzione delle label e dei gruppi.

```
dataset = StandardDataset(
    df_scaled,
    label_name=label_col,
    favorable_classes=[fav_label],
    protected_attribute_names=[prot_attr],
    privileged_classes=[[priv_val]]
)
print("OK: StandardDataset picreato (scalato).")

np.random.seed(SEED)
train, test = dataset.split([0.7], shuffle=True)

print("Distribuzione label TRAIN:", Counter(train.labels.ravel()))
print("Distribuzione label TEST :", Counter(test.labels.ravel()))
```

Figura 3.33: Split German

4. **helper per il calcolo delle metriche:** La figura 3.34 mostra come anche per German è stata implementata una funzione che esegue il training di Prejudice Remover con un dato valore di η , calcola le predizioni e le metriche di fairness.

```
def run_pr_and_metrics(eta: float):
    model = PrejudiceRemover(sensitive_attr=prot_attr, eta=eta)
    model.fit(train)
    pred = model.predict(test)

    met = ClassificationMetric(
        test, pred,
        privileged_groups=[[prot_attr: priv_val]],
        unprivileged_groups=[[prot_attr: unpriv_val]]
    )

    dfp = pred.convert_to_dataframe()[0]
    return {
        "eta": eta,
        "SPD": float(met.statistical_parity_difference()),
        "DI": float(met.disparate_impact()),
        "Consistency": float(met.consistency()),
        "Accuracy": float(met.accuracy()),
        "pred_df": dfp
    }
```

Figura 3.34: Helper e calcolo metriche

3.3.4 Validazione Funzione di Generazione Snippet

Per validare la funzionalità del plugin dedicata alla generazione automatica di snippet di codice, si è fatto riferimento a uno script baseline standard, costruito sulla base del dataset COMPAS. Lo script baseline implementa una pipeline di Machine Learning semplice e trasparente:

- caricamento del dataset pre processato COMPAS
- divisione dei dati in training e test set con proporzione 70/30
- addestramento di un classificatore Logistic Regression con standardizzazione delle feature
- calcolo delle predizioni sul test set
- valutazione dell'accuratezza del modello
- costruzione degli oggetti `BinaryLabelDataset` di AIF360 per rappresentare i dati con le informazioni su label e attributo protetto.
- calcolo delle metriche di fairness, nello specifico Statistical Parity Difference, Disparate Impact e consistency

Partendo da questo codice baseline, il plugin è stato testato tramite la finestra di interazione con l'utente. È stato chiesto al sistema di generare uno snippet di codice personalizzato per integrare l'algoritmo di Prejudice Remover nel codice baseline; il codice prodotto dal bot è stato poi copiato e incollato senza modifiche manuali nel file Python contenente il baseline, subito dopo la sezione dedicata al calcolo delle metriche di fairness. Successivamente, il codice è stato analizzato ed è stato testato.

CAPITOLO 4

Risultati

L'obiettivo dello studio è quello di dimostrare se un plugin integrato nell'ambiente di sviluppo potesse aiutare e supportare gli sviluppatori nella valutazione della fairness e nella mitigazione di bias nei loro modelli di Machine Learning, mettendo a disposizione strumenti di analisi, calcolo e di mitigazione dei bias in maniera semplice, che non comportasse numerose conoscenze dell'argomento. Dai risultati ottenuti si evince che ciò è effettivamente possibile; le varie funzioni implementate infatti hanno mostrato la capacità di identificare e mitigare i bias, restituendo codice più equo.

Nella seguente sezione verranno riportati e discussi i risultati delle sperimentazioni.

4.1 Risultati Per la Funzione di Calcolo

La prima funzionalità analizzata riguarda il calcolo delle metriche, l'obiettivo di questa fase è stato quello di quantificare la presenza di bias nei dataset di riferimento COMPAS e Adult (UCI) mediante il calcolo delle seguenti metriche: Statistical Parity Difference, Disparate Impact e Consistency.

Nella tabella 4.1 vengono riportati i dati di output restituiti dalla funzione di calcolo delle metriche.

Dataset	SPD ATTR1	SPD ATTR2	DI ATTR1	DI ATTR2	Consistency
Adult	-0.1945	-0.1014	0.3596	0.6005	0.7750
COMPAS	-0.1276	-0.0971	0.8029	0.8403	0.6757

Tabella 4.1: valori delle metriche di fairness

I valori ottenuti riportati in tabella sono stati confrontati con i valori dati in output dalla funzione implementata nel plugin, dimostrando la validità del calcolo delle metriche di fairness del software.

4.2 Risultati per la funzione di Reweighing

La seconda funzionalità ad essere stata testata è quella di reweighing, applicata ai dataset Adult e COMPAS. I risultati da confrontare per questo test sono due:

- I valori delle metriche pre e post applicazione della funzione reweighing
- I valori delle metriche di fairness in output dello script eseguito in Python Console e i valori restituiti dalla funzione reweighing implementata dal plugin ed eseguita tramite `actionButton`

I valori delle metriche calcolati nella fase di pre reweighing coincidono con quelli già calcolati e riportati nella tabella 4.1, relativi alla valutazione iniziale dei dataset; di conseguenza, non verranno riportati.

Di seguito la tabella 4.2 riporta tutti i valori delle metriche post reweighing. I risultati mostrano come, dopo l'applicazione della funzione di reweighing, lo

Dataset	SPD ATTR1	SPD ATTR2	DI ATTR1	DI ATTR2
Adult	-0.1127	-0.0080	0.6349	0.9706
COMPAS	-0.7594	-0.0329	0.8691	0.9390

Tabella 4.2: valori delle metriche post reweighing

Statistical Parity Difference si avvicini a zero e il Disparate Impact risalga verso

valori vicini all'unità, dimostrando quindi che la funzione di reweighing agisce perfettamente bilanciando i pesi dei due dataset. Inoltre, i risultati in output della funzione implementata nel plugin combaciano con quelli della Python Console, dimostrando come anche il plugin applichi in modo corretto la funzione.

4.3 Risultati della Funzione Prejudice Remover

Per i test sulla funzionalità del Prejudice remover sono stati usati i dataset COMPAS e German Credit, anche per questa funzione è avvenuto un confronto tra due gruppi di risultati:

- I risultati ottenuti con un $\eta = 0.0$ con i risultati dell'esecuzione della funzione con $\eta = 25.0$ tramite uno script da Python Console
- I risultati ottenuti con $\eta = 0.0$ e $\eta = 25.0$ svolti tramite la funzione implementata nel plugin con i risultati dello script eseguito dalla console.

Di seguito la tabella 4.3 che riporta i valori con $\eta = 0.0$

Dataset	SPD	DI	Consistency	Accuracy
COMPAS	0.2176	1.8883	0.6719	0.3322
German Credit	-0.1879	0.7766	0.7546	0.6133

Tabella 4.3: valori delle metriche ETA = 00

Mentre la tabella 4.4 riporta i valori con $\eta = 25$

Dataset	SPD	DI	Consistency	Accuracy
COMPAS	0.1160	1.3402	0.6719	0.3381
German Credit	-0.0876	0.9297	0.7546	0.6000

Tabella 4.4: valori delle metriche ETA = 25

I risultati evidenziano come, con $\eta = 0.0$, i valori delle metriche rimangano sostanzialmente invariati rispetto al calcolo delle metriche di base, mentre con $\eta = 25$

si denota un chiaro miglioramento delle metriche: con S viene ridotto sensibilmente, DI che si avvicina maggiormente alla soglia 0.9, la Consistency resta stabile e l'Accuracy viene leggermente ridotta, ma è un risultato atteso poiché all'aumento della regolarizzazione tende a ridurre la capacità predittiva del modello.

4.4 Risultati della Generazione di Snippet Personalizzati

La verifica della funzione che genera gli snippet di codice personalizzato è stata fatta tramite una checklist, sono state stabilite delle regole che il codice generato deve rispettare:

- l'import corretto della classe `PrejudiceRemover` dal modulo `aif360.algorithms.inprocessing`
- la costruzione di `BinaryLabelDataset` per il training set e per il test set
- riutilizzo dei nomi già presenti nello script baseline per le variabili
- addestramento del classificatore PR con il metodo `fit()` sul dataset di training
- la produzione delle predizioni mitigate sul dataset di test tramite il metodo `predict()`
- il calcolo delle metriche di fairness post mitigazione, attraverso le API di AIF360
- la stampa delle metriche calcolate

Alla fine del test si è quindi analizzato il codice e l'analisi ha evidenziato come esso rispetti tutte queste regole precedentemente stabilite, dimostrando quindi che il plugin è in grado di creare porzioni di codice adeguate e personalizzate per l'applicazione della funzione Prejudice Remover al codice utente.

4.5 Considerazioni Finali

Dai risultati ottenuti emerge evidente come il plugin sia in grado di svolgere in maniera efficace compiti come l'analisi, il calcolo della fairness e la mitigazione dei bias, confermando la validità dell'approccio adottato.

Risposta RQ 1: Il lavoro svolto ha dimostrato che il plugin è effettivamente in grado di fornire supporto direttamente nell'IDE, difatti le metriche calcolate sono risultate affidabili, le funzioni per la mitigazione dei bias hanno portato a dei chiari miglioramenti nelle metriche relative ai dataset di test e tutto questo senza dover costringere l'utente a lasciare l'ambiente di programmazione.

Risposta RQ 2: Anche in questo caso, i risultati confermano una risposta affermativa alle domande di ricerca, la presenza di pulsanti dedicati, di output tabellari e di funzioni pre implementate ha reso il processo di analisi semplice e immediato. Inoltre, la presenza di un chatbot che può chiarire i dubbi dell'utente e proporre snippet personalizzati per la mitigazione dei bias, ha contribuito a ridurre il carico di lavoro dell'utente.

CAPITOLO 5

Conclusioni

Questo lavoro ha cercato di dimostrare se fosse possibile la creazione di un tool che supporti gli sviluppatori nella creazione di codice equo direttamente nell'IDE di programmazione, si è quindi posto come obiettivo principale la creazione di un plugin per Pycharm e si è dimostrato come questi tool possano effettivamente esistere e aiutare gli sviluppatori. Il tool è stato creato con l'obiettivo di essere un collegamento diretto tra le funzioni contenute nella libreria AIF360 e i programmatori, implementando le funzioni direttamente nell'ambiente di sviluppo evitando allo sviluppatore la necessità di uscire dall'ambiente per recuperare informazioni e documentazioni per l'applicazione di suddette funzioni. In fine il progetto rappresenta solo un primo passo verso strumenti pratici per il supporto alla programmazione di codice equo, sicuramente un futuro ampliamento delle metriche calcolate, delle funzioni supportate e dei miglioramenti relativi alla quality of life del sistema porteranno ad un impatto maggiore del lavoro svolto.

Bibliografia

- [1] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan, “A survey on bias and fairness in machine learning,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 6, pp. 1–35, 2021. (Citato alle pagine iii, 4, 5, 7, 10, 11, 12 e 21)
- [2] O. Hort, A. Jonsson, and L. Carlsson, “Bias mitigation for machine learning classifiers: A comprehensive survey,” *Artificial Intelligence Review*, vol. 55, no. 2, pp. 1407–1476, 2022. (Citato alle pagine 2, 5, 7 e 10)
- [3] S. Caton and C. Haas, “Fairness in machine learning: A survey,” *arXiv preprint arXiv:2010.04053*, 2020. (Citato alle pagine 2, 4, 5, 10, 11, 12 e 21)
- [4] M. Hardt, E. Price, and N. Srebro, “Equality of opportunity in supervised learning,” *Advances in Neural Information Processing Systems*, vol. 29, 2016. (Citato alle pagine 4, 10 e 12)
- [5] C. Dwork, M. Hardt, T. Pitassi, O. Reingold, and R. S. Zemel, “Fairness through awareness,” *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pp. 214–226, 2012. (Citato alle pagine 4 e 11)
- [6] S. Barocas, M. Hardt, and A. Narayanan, “Fairness and machine learning,” 2019, book draft available at <https://fairmlbook.org>. (Citato alle pagine 4, 6, 7 e 21)
- [7] J. Angwin, J. Larson, S. Mattu, and L. Kirchner, “Machine bias: There’s software used across the country to predict future criminals. and it’s biased against blacks,”

- ProPublica*, 2016, available at <https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing>. (Citato alle pagine 6 e 21)
- [8] S. Hooker, A. Courville, G. Clark, Y. Dauphin, and A. Frome, “Characterising bias in compressed models,” *arXiv preprint arXiv:1905.00515*, 2021. (Citato a pagina 6)
- [9] E. S. Jo and T. Gebru, “Lessons from archives: strategies for collecting sociocultural data in machine learning,” *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pp. 306–316, 2020. (Citato a pagina 6)
- [10] H. Suresh and J. V. Guttag, “A framework for understanding unintended consequences of machine learning,” *Communications of the ACM*, vol. 64, no. 2, pp. 62–71, 2019. (Citato a pagina 6)
- [11] F. Kamiran and T. Calders, “Classifying without discriminating,” *International Conference on Computer, Control and Communication*, pp. 1–6, 2009. (Citato alle pagine 7 e 10)
- [12] F. Kamiran, A. Karim, and X. Zhang, “Quantifying explainable discrimination and its application to fairness-enhancing classification,” *Knowledge and Information Systems*, vol. 35, no. 3, pp. 613–644, 2013. (Citato alle pagine 7 e 10)
- [13] M. Feldman, S. A. Friedler, J. Moeller, C. Scheidegger, and S. Venkatasubramanian, “Certifying and removing disparate impact,” in *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. ACM, 2015, pp. 259–268. (Citato alle pagine 8, 10, 11 e 21)
- [14] R. Zemel, Y. Wu, K. Swersky, T. Pitassi, and C. Dwork, “Learning fair representations,” in *International Conference on Machine Learning*. PMLR, 2013, pp. 325–333. (Citato a pagina 8)
- [15] R. K. Bellamy, K. Dey, M. Hind, S. Hoffman, S. Houde, K. N. R. Kannan, P. Lohia, J. Martino, S. Mehta, A. Mojsilovic *et al.*, “Ai fairness 360: An extensible toolkit for detecting, understanding, and mitigating unwanted algorithmic bias,” *IBM Journal of Research and Development*, vol. 63, no. 4/5, pp. 4–1, 2019. (Citato alle pagine 9 e 21)

-
- [16] T. Kamishima, S. Akaho, H. Asoh, and J. Sakuma, "Fairness-aware classifier with prejudice remover regularizer," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2012, pp. 35–50. (Citato alle pagine 10 e 11)
- [17] B. H. Zhang, B. Lemoine, and M. Mitchell, "Mitigating unwanted biases with adversarial learning," in *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*. ACM, 2018, pp. 335–340. (Citato a pagina 10)
- [18] S. A. Friedler, C. Scheidegger, S. Venkatasubramanian, S. Choudhary, E. P. Hamilton, and D. Roth, "A comparative study of fairness-enhancing interventions in machine learning," *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pp. 329–338, 2019. (Citato alle pagine 12 e 21)
- [19] J. Kleinberg, S. Mullainathan, and M. Raghavan, "Inherent trade-offs in the fair determination of risk scores," *arXiv preprint arXiv:1609.05807*, 2016. (Citato a pagina 12)
- [20] R. Kohavi, "Scaling up the accuracy of naive-bayes classifiers: a decision-tree hybrid," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*. AAAI Press, 1996, pp. 202–207. (Citato a pagina 21)
- [21] A. Sharma, S. Jung, M. Glass *et al.*, "Certifai: A comprehensive bias detection and mitigation toolkit," in *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*. ACM, 2020, pp. 166–172. (Citato a pagina 21)
- [22] A. Chouldechova, "Fair prediction with disparate impact: A study of bias in recidivism prediction instruments," *Big data*, vol. 5, no. 2, pp. 153–163, 2017. (Citato a pagina 21)
- [23] J. Dressel and H. Farid, "The accuracy, fairness, and limits of predicting recidivism," *Science Advances*, vol. 4, no. 1, p. eaao5580, 2018. (Citato a pagina 21)
- [24] U. Grömping, "South german credit data: Correcting a widely used data set," *Report 4/2019, Department of Statistics, Freie Universität Berlin*, 2019. (Citato a pagina 21)

-
- [25] M. Donini, L. Oneto, S. Ben-David, J. Shawe-Taylor, and M. Pontil, "Empirical risk minimization under fairness constraints," in *Advances in Neural Information Processing Systems*, vol. 31, 2018. (Citato a pagina 21)
- [26] B. Bruegge and A. H. Dutoit, "Object-oriented software engineering. using uml, patterns, and java," *Learning*, vol. 5, no. 6, p. 7, 2009.
- [27] V. Garousi, M. Felderer, and M. V. Mäntylä, "Guidelines for including grey literature and conducting multivocal literature reviews in software engineering," *Information and Software Technology*, vol. 106, pp. 101–121, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584918301939>
- [28] V. Pontillo, F. Palomba, and F. Ferrucci, "Static test flakiness prediction: How far can we go?" *Empirical Software Engineering*, vol. 27, no. 7, pp. 1–44, 2022.
- [29] M. De Stefano, F. Pecorelli, D. A. Tamburri, F. Palomba, and A. De Lucia, "Splitting community patterns and smells: A preliminary study," in *Proceedings of the IEEE/ACM 42nd international conference on software engineering workshops*, 2020, pp. 703–710.
- [30] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, and F. Palomba, "The secret life of software vulnerabilities: A large-scale empirical study," *IEEE Transactions on Software Engineering*, 2022.
- [31] G. Giordano, F. Palomba, and F. Ferrucci, "On the use of artificial intelligence to deal with privacy in iot systems: A systematic literature review," *Journal of Systems and Software*, p. 111475, 2022.
- [32] F. Casillo, V. Deufemia, and C. Gravino, "Detecting privacy requirements from user stories with nlp transfer learning models," *Information and Software Technology*, vol. 146, p. 106853, 2022.
- [33] S. Lambiase, G. Catolino, D. A. Tamburri, A. Serebrenik, F. Palomba, and F. Ferrucci, "Good fences make good neighbours? on the impact of cultural and geographical dispersion on community smells," 2022.

- [34] G. Sellitto, E. Iannone, Z. Codabux, V. Lenarduzzi, A. De Lucia, F. Palomba, and F. Ferrucci, "Toward understanding the impact of refactoring on program comprehension," in *29th International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 2022, pp. 1–12.
- [35] M. J. Kusner, J. Loftus, C. Russell, and R. Silva, "Counterfactual fairness," in *Advances in neural information processing systems*, vol. 30, 2017.

Ringraziamenti

Desidero esprimere la mia gratitudine al Professor Fabio Palomba, al dott. Gianmario Voira e alla dott.ssa Alessandra Parziale per il supporto durante la stesura della tesi.

Vorrei poi ringraziare la mia famiglia per aver messo sempre al primo posto la mia istruzione, per avermi sempre concesso piena libertà sul metodo di studio e sulle scelte del mio percorso formativo. Se oggi ho conseguito questo titolo, è soprattutto grazie alle tante volte in cui mi avete ripetuto di dover studiare unicamente per me stesso.

Vorrei inoltre ringraziare il gruppo Patatine, è stato grazie a voi ragazzi se ho vissuto gli ultimi anni dell'università con grande serenità, anche se ognuno ha continuato sul proprio percorso mi auguro di continuare a sostenerci a vicenda come abbiamo sempre fatto, e che tutti voi possiate raggiungere gli obiettivi che vi siete prefissati.

In fine, un doveroso e grandissimo ringraziamento va alla mia ragazza Alessia, grazie per essere sempre la mia sostenitrice numero uno, per essere sempre quella che crede in me sia quando le cose vanno male che quando vanno bene; il mio augurio è di continuare a poter contare l'uno sull'altra, e di vederci un giorno realizzati insieme nonostante tutte le difficoltà che si presenteranno sui nostri percorsi.